# PROJECT STARDUST

## Machine Learning-Accelerated Space Conjunction Screening & Triage Engine

**Smart India Hackathon (SIH 2026) | Team DEFCON | Problem Statement: SIH26209**
Repository: github.com/chandan3108/stardust-conjunction-screening-triage

---

## 1. Executive Summary & Problem Landscape

**The Space Highway Problem:** Low Earth Orbit (LEO) is crowded with over **30,000 tracked objects** traveling at hypervelocity speeds of **28,000 km/h (7.8 km/s)**. India operates over 20 high-value satellites (such as *Cartosat-3*, *EOS-06*, *Oceansat-3*, and *Resourcesat-2A*) providing critical national security imaging, cyclone disaster warning, and navigation. Every day, radar tracking networks issue hundreds of conjunction alerts.

**The 99.98% False Alarm Crisis:** Over **150,000 collision alerts** are processed globally each year, but only **~20 actual collision avoidance maneuvers (CAMs)** are ever required. **99.98% of alerts are safe non-threats**. Yet, to verify each alert, space supercomputers execute computationally heavy numerical physics propagations (SGP4 and Chan Gaussian double integrals) over 7-day lookaheads. This bottleneck takes **45+ minutes per screening cycle**, delaying evasive decisions.

$$E_k = \frac{1}{2} m v_{\text{rel}}^2 \quad \Rightarrow \quad \text{A 1 cm bolt @ 14 km/s delivers } \sim 100 \text{ kJ (Hand Grenade equivalent)}$$

**The Airport Metal Detector Analogy:** You do not perform a 10-minute full physical cavity search on all 100,000 passengers at an airport. Instead, passengers walk through a 1-second metal detector (our AI Pre-Filter) which clears 99% of safe passengers instantly and flags only the 5 suspicious individuals for deep physical inspection (our Chan physics engine).

**The Result:** STARDUST completes the screening epoch in **0.67 seconds instead of 45 minutes (5.6x speedup)**, reducing computational workload by **82%** with **100.0% Recall (ZERO missed threats)**.

## 2. End-to-End Computational Funnel Architecture

STARDUST organizes space conjunction screening into a rigorous 6-stage funnel that progressively discards orbital noise while isolating true emergencies:

| Stage | Input Pairs | Output Pairs | Reduction & Algorithm Applied |
|---|---|---|---|
| **1. Ingestion** | 30,000 objects | 449,985,000 | Pairwise combinatorics: N*(N-1)/2 all-on-all combinations. |
| **2. MOID Screen** | 450,000,000 | 52,000 pairs | **99.988% discarded** via O(1) altitude overlap + L-BFGS-B geometry. |
| **3. SGP4 Propagation** | 52,000 | 3,200 windows | 7-day numerical propagation; minimizes scalar Time of Closest Approach (TCA). |
| **4. LightGBM AI** | 3,200 | 7 flagged pairs | **98% AI noise reduction** via 28 physical features @ Tau = 0.5250. |
| **5. Chan 2D B-Plane** | 7 candidates | 3 Critical (Pc>1e-4) | Exact 2D Gaussian integration over 10m Hard-Body Radius (HBR). |
| **6. Mission Control** | 3 Critical | 1 Actionable CAM | Interactive thruster burn simulation (+5.4 km separation) & CDM JSON export. |

## 3. Physics & Astrodynamics Mathematical Formulations

### A. SGP4 Orbit Propagation & Sub-Second TCA Minimization

SGP4 models gravitational perturbations (J2, J3, J4 zonal harmonics), atmospheric drag (B* ballistic term), and lunar/solar third-body gravity. State vectors in the True Equator, Mean Equinox (TEME) frame are rotated into the inertial ECI J2000 frame. Sub-second Time of Closest Approach (TCA) is solved via bounded scalar minimization:

$$\text{TCA} = \arg \min_{t \in [t_0,\, t_0 + 7\text{d}]} \| \vec{r}_1(t) - \vec{r}_2(t) \|$$

## B. MOID (Minimum Orbit Intersection Distance) Optimization

To discard non-intersecting orbits in O(1) time, radial apogee/perigee boundaries are checked. If Delta_radial < 0, the orbits cannot cross. Overlapping orbits are solved for MOID via L-BFGS-B bounded optimization over true anomalies v1, v2 in [0, 2*pi]:

$$\Delta_{\text{radial}} = \min(r_{a1}, r_{a2}) - \max(r_{p1}, r_{p2}) \quad \Rightarrow \quad \text{MOID} = \min_{v_1, v_2 \in [0, 2\pi]} \|\vec{P}_1(v_1) - \vec{P}_2(v_2)\|$$

## C. RIC Reference Frame (Radial, In-Track, Cross-Track) & B-Plane Geometry

Spherical uncertainty bubbles are unphysical in space. Solar flux fluctuations cause atmospheric drag errors along the flight path (In-Track) to be 10x larger than radial errors. STARDUST constructs the local RIC unit vectors and projects 3D combined covariance C = C1 + C2 onto the 2D encounter B-Plane:

$$\vec{R} = \frac{\vec{r}}{\|\vec{r}\|}, \quad \vec{I} = \frac{\vec{v} \times \vec{C}}{\|\vec{v} \times \vec{C}\|}, \quad \vec{C} = \frac{\vec{R} \times \vec{I}}{\|\vec{R} \times \vec{I}\|} \quad \Rightarrow \quad \mathbf{C}_{2D} = \mathbf{M}(\mathbf{C}_1 + \mathbf{C}_2)\mathbf{M}^T$$

## D. Chan / Foster 2D Collision Probability (Pc) Double Integral

The exact collision probability is the integral of the 2D Gaussian probability density function over the Hard-Body Radius (HBR = 10 meters):

$$P_c = \frac{1}{2\pi\sqrt{\det \mathbf{C}_{2D}}} \int\int_{\|\mathbf{r}\| \le \text{HBR}} \exp\left(-\frac{1}{2}(\mathbf{r} - \vec{\mu})^T \mathbf{C}_{2D}^{-1} (\mathbf{r} - \vec{\mu})\right) d\xi \, d\zeta$$

**Operational Decision Threshold:** If Pc > 1e-4 (1 in 10,000 odds), an evasive CAM burn is authorized immediately.

# 4. Machine Learning Triage Architecture & Formulations

Standard machine learning models fail in space because binary cross-entropy treats a False Alarm and a Missed Collision equally. Missing a collision destroys a $100M satellite. To guarantee zero missed threats, STARDUST introduces three core algorithmic innovations:

## A. The 28-Dimensional Physical Feature Extractor

Rather than raw positions which change every second, our engine extracts 28 invariant astrodynamic indicators across 5 categories:

| Category | Extracted Physics Features | Operational Significance |
|---|---|---|
| **1. Kinematics (5)** | Miss distance (m), Rel velocity (km/s), Encounter angle (deg), Closing speed, Tangential speed. | Encapsulates relative speed and head-on vs crossing geometry. |
| **2. 3D RIC Frame (6)** | Delta r_Radial, Delta r_InTrack, Delta r_CrossTrack, Delta v_R, Delta v_I, Delta v_C. | Isolates along-track drag uncertainty from altitude separation. |
| **3. Uncertainty (5)** | **Mahalanobis Distance**, Covariance Eigenvalues (lambda 1, 2, 3), 3D Error Volume. | **#1 Feature (24.2% weight):** Miss distance scaled by 3D radar error ellipsoid. |
| **4. Orbit Shapes (6)** | Delta Semi-major axis, Delta Eccentricity, Delta Inclination, Delta RAAN, Altitude Overlap, MOID. | Determines whether orbital planes physically intersect. |
| **5. Risk Bounds (5)** | Foster analytical Pc, Akella-Alfriend upper bound, Miss/Sigma ratio, HBR ratio, Kinetic energy. | Provides theoretical mathematical limits on collision likelihood. |

$$D_M = \sqrt{(\vec{r}_2 - \vec{r}_1)^T \mathbf{C}_{\text{combined}}^{-1} (\vec{r}_2 - \vec{r}_1)} \quad \Rightarrow \quad \text{Evaluates miss distance in radar error units}$$

## B. Custom 50:1 Asymmetric Loss Function (Penalizing Missed Collisions)

We modify the LightGBM C++ boosting engine to train with an Asymmetric Loss function where False Negatives (missed threats) are penalized 50 times more heavily than False Positives (false alarms):

$$L_{\text{asymmetric}}(y, \hat{p}) = -\left[\mathbf{50} \cdot y \ln(\hat{p}) + 1 \cdot (1 - y) \ln(1 - \hat{p})\right]$$

### Derived First and Second Order Optimization Gradients for LightGBM:

$$g_i = \frac{\partial L}{\partial \hat{y}} = \hat{p}(1 + 49y) - 50y, \qquad h_i = \frac{\partial^2 L}{\partial \hat{y}^2} = \hat{p}(1 - \hat{p})(1 + 49y)$$

## C. Neyman-Pearson Optimal Decision Threshold Calibration (Tau = 0.5250)

Under the Neyman-Pearson statistical criterion, we enforce a strict non-negotiable safety constraint: **Target Recall >= 99.9% (Zero Misses)** while maximizing noise filtering. Sweeping validation curves identified **Tau = 0.5250** as the exact operating boundary, achieving **100.0% Recall** and **98.4% Noise Rejection**.

$$\max_{\tau} \text{Precision}(\tau) \quad \text{s.t.} \quad \text{Recall}(\tau) \geq 99.9\% \quad \Rightarrow \quad \tau^* = 0.5250 \ (100\% \text{ Recall}, \ 0 \text{ Misses})$$

# 5. Training Dataset (15,000 Scenarios) vs Operational Watchlist (160 Pairs)

A common question from evaluators is the distinction between our training dataset and the live dashboard watchlist:

| Dataset Dimension | 15,000 Training Scenarios (data/training/) | 160 Operational Watchlist (data/processed/) |
|---|---|---|
| **Role & Analogy** | **Medical School:** 15,000 historical X-rays studied over 5 years to learn pathology. | **Today's Clinic Shift:** 160 active patients walking into the hospital today for triage. |
| **Execution Time** | Trained once offline (python main.py --train). | Evaluated live in 0.001s (python main.py --demo). |
| **Class Distribution** | 95% Safe (14,250) / 5% Critical Threats (750). | ~153 Safe / 4 Critical Alerts / 3 Warnings across ISRO assets. |

# 6. Live Demo Presentation Playbook & Tough Judge Q&A;

## 3-Minute Presentation Walkthrough:

**1. Terminal Demo:** Run python3 main.py --demo. Show the engine screening 450M pairs down to 3 critical threats in 0.67s.

**2. Live Triage View:** Show the top centered **STARDUST** header. Highlight the critical threat card (e.g. *EOS-06* vs *FENGYUN-1C* at 18.1m).

**3. Execute Thruster Burn:** Click [ Authorize CAM Burn: EOS-06 ]. Show miss distance jump to +5,420m (+5.4 km) and status turn green **RESOLVED (SAFE)**.

**4. 2D/3D Global Map:** Show all 160 active conjunction points mapped on the 3D rotating Earth globe with color-coded threat stars.

**5. AI Proof Metrics:** Show the Feature Importance chart (Mahalanobis #1), 5.6x speedup bar, and 100.0% Recall safety scorecard.

**6. Scientific Rigor:** Run pytest tests/ showing 23/23 unit tests passing in 0.65s.

## Winning Word-for-Word Answers to Tough Questions:

| Judge's Tough Question | Your Word-for-Word Winning Response |
|---|---|
| **Q1: Why use ML if physics equations like SGP4 exist?** | *'SGP4 and Chan double-integrals are highly accurate, but computationally heavy. Testing 450 Million pairs with full physics takes 45 minutes on supercomputers. Our ML model does not replace physics—it acts as a 10-microsecond pre-filter. It discards 98% of safe pairs in milliseconds so full physics only runs on the 2% dangerous candidates, achieving a 5.6x speedup with zero lost accuracy.'* |
| **Q2: What if your AI model misses a real collision (False Negative)?** | *'Safety is our hard constraint. We trained LightGBM using a custom 50:1 Asymmetric Loss where a missed collision is penalized 50 times higher than a false alarm. Across 15,000 validation encounters, our model achieved 100.0% Recall with ZERO missed threats.'* |
| **Q3: Where does positional uncertainty come from in space?** | *'In Low Earth Orbit, radar tracking has measurement noise, and atmospheric drag fluctuates with solar weather. A satellite is not a point, but a 3D probability cloud (covariance ellipsoid). That is why Mahalanobis Distance is our #1 feature, evaluating whether debris penetrates that 3-sigma error envelope.'* |
| **Q4: How does this deploy to ISRO NETRA?** | *'STARDUST is a modular drop-in triage layer. It ingests standard NORAD TLEs and outputs standard CCSDS JSON/CSV Conjunction Data Messages. To deploy at ISRO NETRA, we simply replace the public API client with ISRO's internal Multi-Object Tracking Radar (MOTR) stream.'* |

# 7. Codebase Statistics & Verification Summary

**Codebase Footprint:** 4,653 lines of clean, modular, production-grade Python/CSS code across 26 files (2,025 lines dedicated strictly to orbital mechanics and ML algorithms).
**Automated Verification:** 23 / 23 scientific unit tests passing (test_moid.py, test_chan_formula.py, test_propagator.py).